

algoritmos

Leslie Garcia

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:
##HECHO EN CLASE ## Primeros comandos en R

R es muy parecido a matlab y permite el trabajo con matrices

```
A<-38
A<-40
B<-60
C<-B-A
C
```

```
## [1] 20
```

```
summary(cars)
```

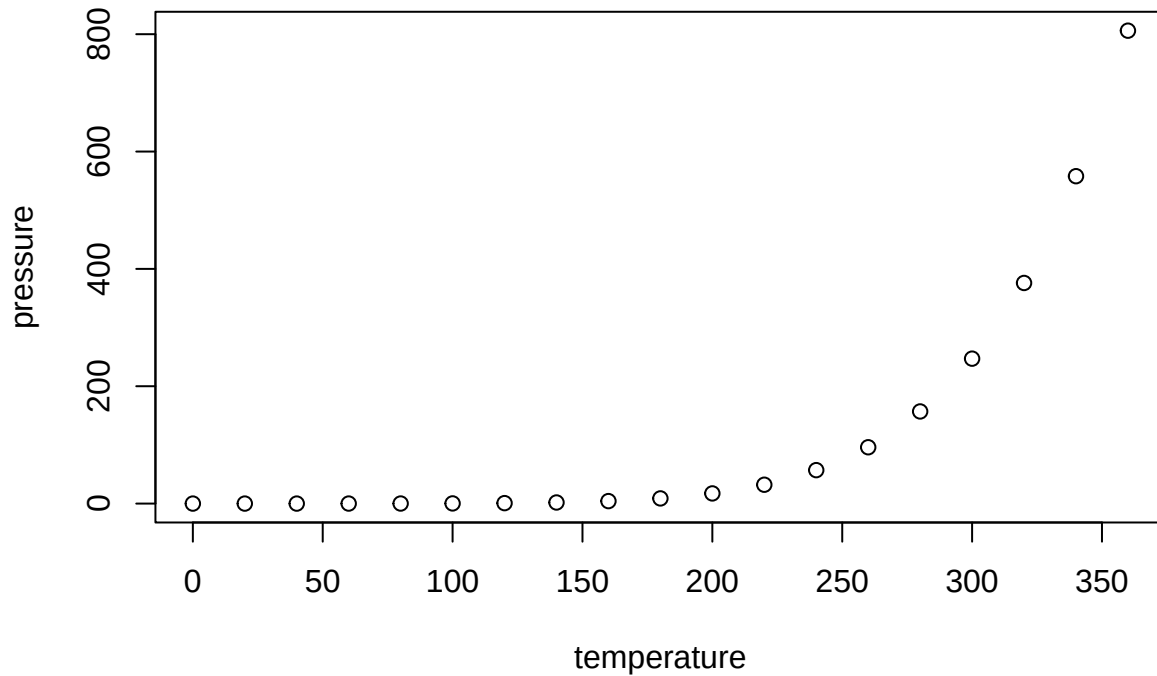
```
##      speed      dist
##  Min.   : 4.0    Min.   : 2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean   : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.   :120.00
```

uso de datasets o base de datos internos de R

Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dado en la tabla de las cuales podemos elegir columnas específicas si escribimos \$ despues de el comando de informacion especifico

Including Plots

You can also embed plots, for example:



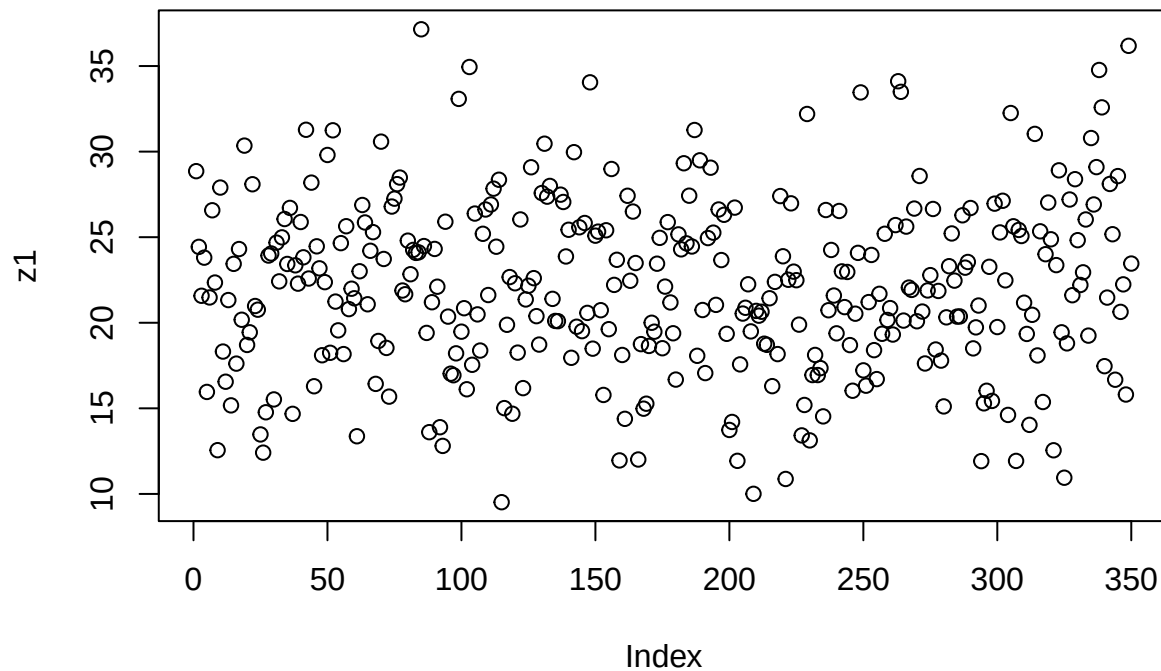
funciones estadísticas

```
z1<-rnorm(350,22,5)
z1
```

```
## [1] 28.857742 24.435043 21.577359 23.805079 15.958145 21.478731 26.572540
## [8] 22.349501 12.555603 27.899486 18.317795 16.555865 21.326437 15.173092
## [15] 23.442316 17.613223 24.306991 20.175984 30.350459 18.704391 19.443316
## [22] 28.092290 20.978355 20.763428 13.475679 12.410610 14.770265 23.927760
## [29] 24.048080 15.520525 24.682083 22.418760 24.989276 26.061918 23.435590
## [36] 26.710693 14.678088 23.356576 22.285285 25.886094 23.823131 31.276252
## [43] 22.585947 28.188372 16.290283 24.465256 23.179995 18.102253 22.373435
## [50] 29.804249 18.245749 31.244633 21.237454 19.556830 24.652114 18.170008
## [57] 25.640155 20.797674 21.989351 21.430629 13.369757 23.008701 26.881102
## [64] 25.859722 21.080054 24.206306 25.293913 16.430004 18.936727 30.583285
## [71] 23.727238 18.529392 15.684269 26.790844 27.252033 28.099748 28.483239
## [78] 21.883602 21.666063 24.796980 22.836423 24.257004 24.081766 24.114239
## [85] 37.146054 24.481089 19.407070 13.607101 21.200713 24.312789 22.106568
## [92] 13.897208 12.806163 25.903231 20.359668 17.025897 16.934305 18.213188
## [99] 33.080789 19.480104 20.851217 16.115198 34.942741 17.544342 26.380508
## [106] 20.490168 18.376214 25.197558 26.617433 21.621123 26.910620 27.836106
## [113] 24.444390 28.356338 9.516941 15.012674 19.882297 22.669135 14.683987
## [120] 22.307381 18.254967 26.036203 16.181034 21.362852 22.160831 29.085257
## [127] 22.606826 20.386461 18.726039 27.579254 30.463040 27.368810 27.994966
## [134] 21.395791 20.106855 20.093825 27.485540 27.056763 23.873450 25.433560
## [141] 17.952743 29.967319 19.768707 25.578923 19.510597 25.829196 20.576883
## [148] 34.047480 18.487727 25.088571 25.323764 20.733873 15.782525 25.403981
## [155] 19.625273 28.983815 22.213412 23.678423 11.957462 18.118047 14.386639
## [162] 27.419428 22.455086 26.497223 23.490431 12.011120 18.751805 14.981150
## [169] 15.272540 18.645924 20.000138 19.477769 23.452214 24.960996 18.512818
## [176] 22.113613 25.880759 21.186093 19.386664 16.679604 25.163107 24.289773
## [183] 29.318303 24.639185 27.424172 24.455937 31.261798 18.070112 29.491677
```

```
## [190] 20.745583 17.060047 24.948913 29.060111 25.264313 21.045526 26.618946
## [197] 23.663588 26.309460 19.354213 13.736638 14.205271 26.728713 11.933466
## [204] 17.568530 20.537774 20.875606 22.248894 19.495238 10.006220 20.694320
## [211] 20.414165 20.643911 18.776637 18.710607 21.432375 16.297251 22.389169
## [218] 18.170328 27.401951 23.885117 10.873758 22.511368 26.977289 22.987458
## [225] 22.486928 19.891683 13.419137 15.196783 32.197502 13.127853 16.945376
## [232] 18.119707 16.945128 17.349884 14.530927 26.587546 20.731446 24.254152
## [239] 21.590559 19.381132 26.532299 23.001911 20.915539 22.968021 18.695094
## [246] 16.028884 20.529863 24.083719 33.459101 17.218770 16.322887 21.207173
## [253] 23.962818 18.400255 16.708014 21.690358 19.356808 25.199088 20.163282
## [260] 20.856752 19.312370 25.708244 34.111150 33.492276 20.128669 25.619246
## [267] 22.056222 21.919432 26.671644 20.088000 28.575239 20.655162 17.620610
## [274] 21.888227 22.783060 26.648067 18.437604 21.858761 17.789512 15.112708
## [281] 20.316206 23.305580 25.217898 22.462122 20.359867 20.369622 26.279445
## [288] 23.220050 23.551627 26.704650 18.502748 19.731002 21.007270 11.916003
## [295] 15.292023 16.027232 23.269415 15.433061 26.959971 19.749715 25.277519
## [302] 27.128494 22.478292 14.616128 32.254152 25.636673 11.928552 25.423482
## [309] 25.068129 21.172173 19.347972 14.035252 20.464310 31.034392 18.090575
## [316] 25.342772 15.369692 24.006891 27.024022 24.877131 12.549178 23.368859
## [323] 28.906165 19.442541 10.946937 18.796832 27.201015 21.610597 28.388976
## [330] 24.825165 22.208009 22.946970 26.034513 19.248864 30.792401 26.912506
## [337] 29.094759 34.769248 32.585340 17.464124 21.471916 28.110596 25.173128
## [344] 16.672495 28.569553 20.636656 22.243835 15.807364 36.180760 23.458935
```

```
plot(z1)
```



```
w1<-length(z1)
```

```
w1
```

```
## [1] 350
```

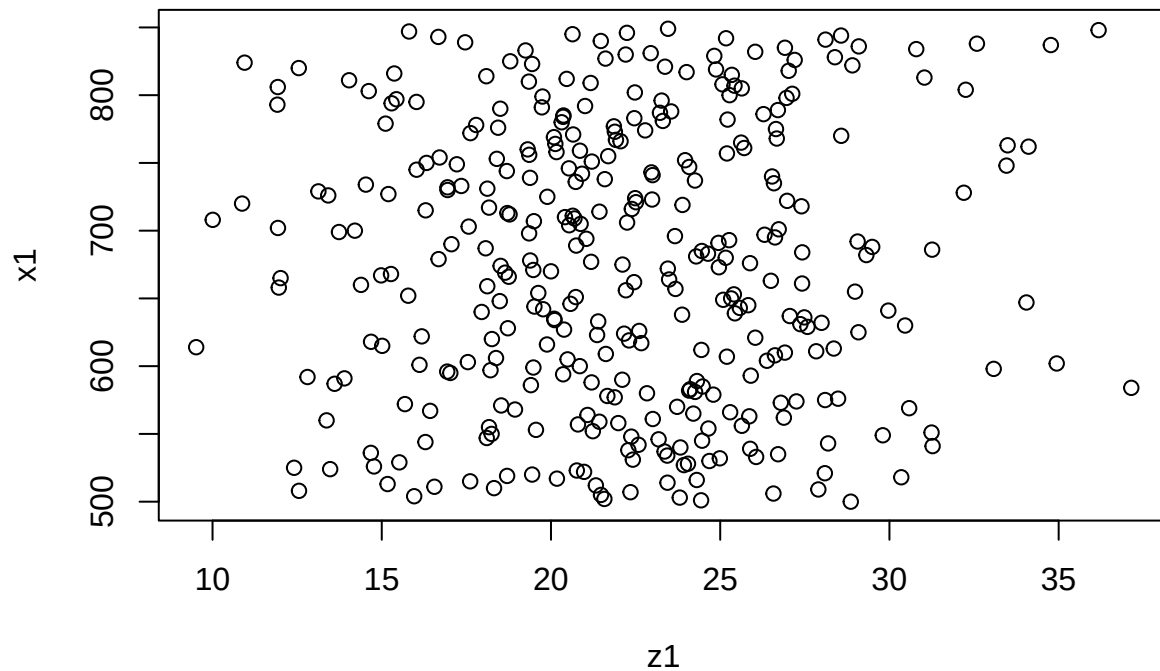
```
x1<-(500:849)
```

```
x1
```

```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
```

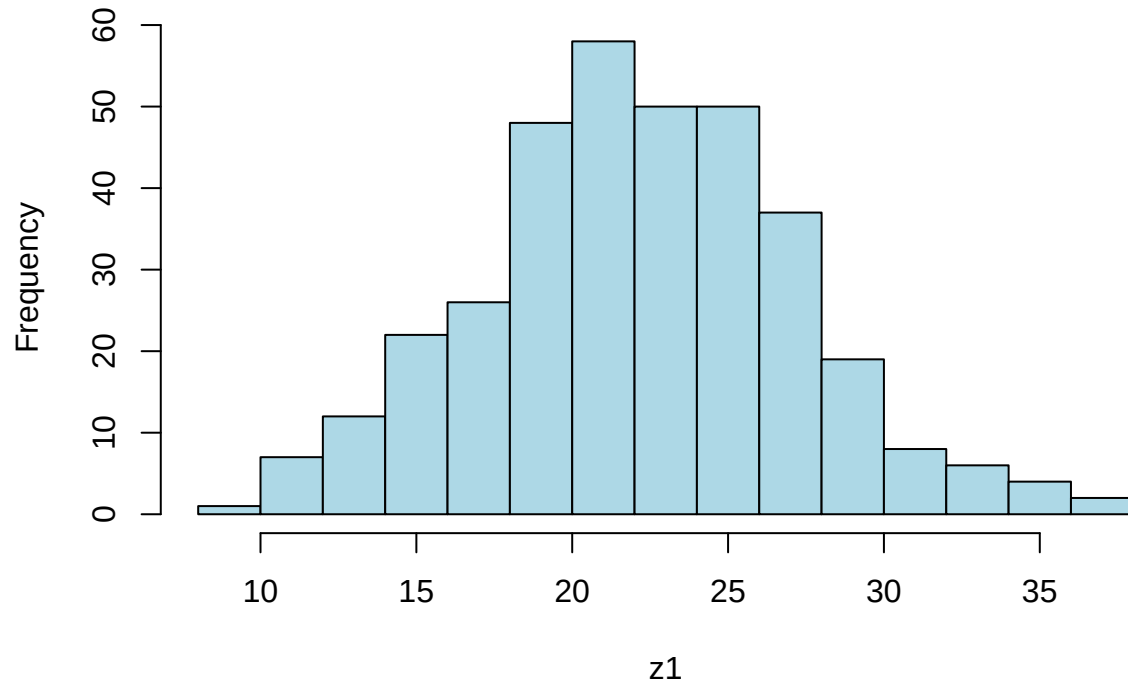
```
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(z1,x1)
```



```
hist(z1, main="Histograma de z1", col="lightblue")
```

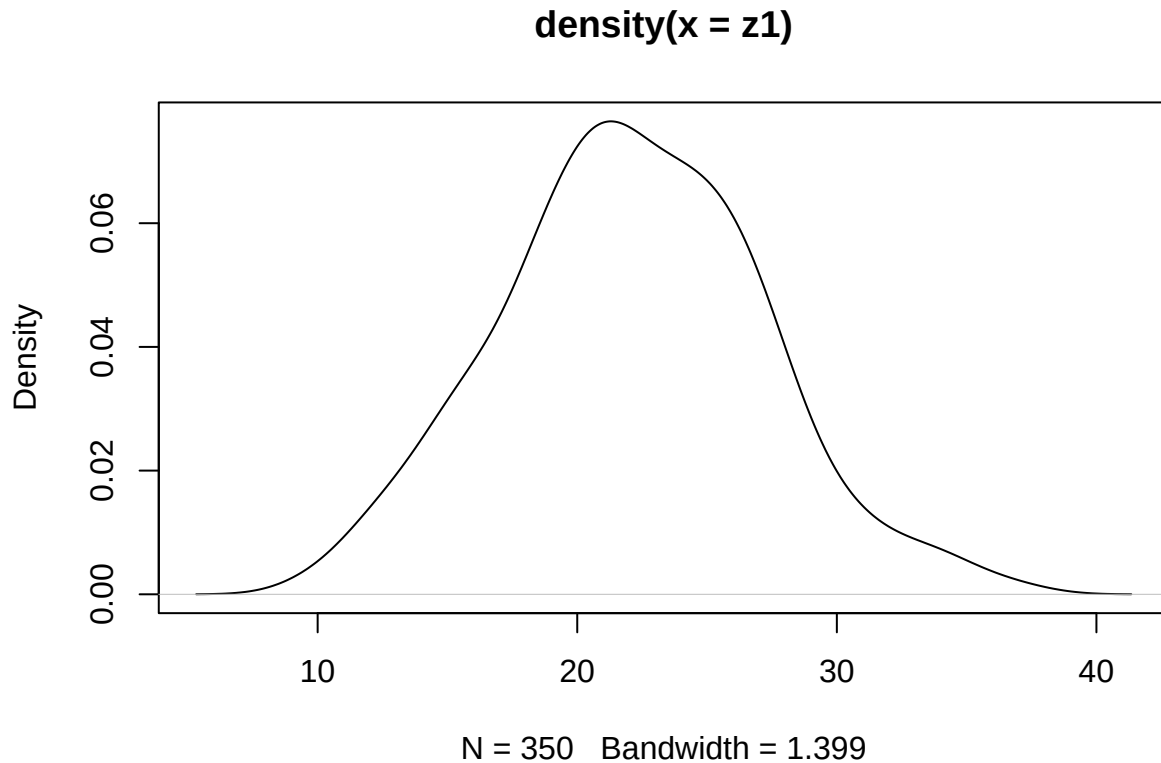
Histograma de z1



```
density(z1)
```

```
##
## Call:
## density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.399
##
##      x          y
## Min.   : 5.32   Min.   :1.005e-05
## 1st Qu.:14.33   1st Qu.:3.235e-03
## Median :23.33   Median :1.652e-02
## Mean   :23.33   Mean    :2.771e-02
## 3rd Qu.:32.34   3rd Qu.:5.296e-02
## Max.   :41.34   Max.    :7.647e-02
```

```
plot(density(z1))
```



```
## Pedir lo de aca
```

Ejercicio 1 de algoritmos

Consigna: Las ultimas 3 cifras de mi DNI son 126, consigna crear una variable que tenga ese número

```
DNI=126
```

consigna: crear un vector que tenga todos los números desde el 1 hasta el 126

```
SECUENCIADNI<-(1:126)
SECUENCIADNI
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
```

Ejercicio 2

Calcular la suma de todos los valores del vector SECUENCIADNI

```
TOTAL<-0
VALORFINAL<-length(SECUENCIADNI)
for (i in 1:VALORFINAL){
```

```
TOTAL<-TOTAL+i }  
TOTAL
```

```
## [1] 8001
```

##Ejercicio 3 ##Repetir el ejercicio anterior pero en Python

```
suma=0  
for i in range (1,127):  
    # range llega hasta uno antes del final, por eso 127  
    suma+=i # <--- Esta línea DEBE tener un espacio al principio  
print(suma)
```

```
## 8001
```

##Ejercicio 4 ##Hacer lo mismo en Julia ## no funciona ## hacer las consignas del pdf en github del profe, en industrial

##HASTA ACÁ LLEGA LO HECHO EN CLASE

1.4 Generar un vector secuencia

```
# Método con for  
A <- 0  
start_for <- Sys.time()  
for (i in 1:50000) { A[i] <- (i*2) }  
end_for <- Sys.time()  
tiempo_for <- end_for - start_for
```

```
# Método con seq() de R  
start_seq <- Sys.time()  
B <- seq(2, 100000, by = 2)  
end_seq <- Sys.time()  
tiempo_seq <- end_seq - start_seq
```

```
# Mostrar resultados  
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(A)
```

```
## [1] 99990 99992 99994 99996 99998 100000
```

```
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(B)
```

```
## [1] 99990 99992 99994 99996 99998 100000
```

```
# Comparar tiempos  
print(paste('Tiempo for:', tiempo_for))
```

```
## [1] "Tiempo for: 0.0149736404418945"
```

```
print(paste('Tiempo seq:', tiempo_seq))
```

```
## [1] "Tiempo seq: 0.00235176086425781"
```

1.5 Implementación de una serie Fibonacci o Fibonacci

```

fibonacci <- function(n) {
  fib <- c(0, 1)
  while (tail(fib, 1) < n) {
    len <- length(fib)
    fib <- c(fib, fib[len] + fib[len - 1])
  }
  return(fib)
}

# Generar hasta superar 1.000.000
resultado <- fibonacci(1000000)
print(resultado)

## [1]      0      1      1      2      3      5      8     13     21
## [10]     34     55     89    144    233    377    610    987   1597
## [19]   2584   4181   6765  10946  17711  28657  46368  75025 121393
## [28] 196418 317811 514229 832040 1346269

print(paste('Iteraciones necesarias:', length(resultado)))

## [1] "Iteraciones necesarias: 32"

print(paste('Ultimo numero mayor a 1.000.000:', tail(resultado, 1)))

## [1] "Ultimo numero mayor a 1.000.000: 1346269"

1.7 Ordenación de un vector por método burbuja

# Cargamos la librería para medir performance
library(microbenchmark)

# 1. Creamos la muestra de tamaño 20.000
x <- sample(1:100000, 20000)

# 2. Definimos la función de la burbuja
burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n - 1)) {
    for (i in 1:(n - j)) {
      if (x[i] > x[i + 1]) {
        temp <- x[i]
        x[i] <- x[i + 1]
        x[i + 1] <- temp
      }
    }
  }
  return(x)
}

# 3. Comparamos la performance
comparacion <- microbenchmark(
  Metodo_Burbuja = burbuja(x),
  Metodo_Sort_R = sort(x),
  times = 1
)

```

```
# 4. Mostramos el resultado
```

```
print(comparacion)
```

```
## Unit: milliseconds
```

```
##      expr      min      lq      mean      median
## Metodo_Burbuja 20928.958671 20928.958671 20928.958671 20928.958671
## Metodo_Sort_R   1.085901    1.085901    1.085901    1.085901
##      uq      max neval
## 20928.958671 20928.958671    1
##    1.085901    1.085901    1
```

1.8 La penitencia de Newton

```
# Metodo 1: suma con for
```

```
metodo_for <- function() {
  numeros <- 1:1e10
  suma <- 0
  for (i in numeros) {
    suma <- suma + i
  }
  return(suma)
}
```

```
# Metodo 2: formula matematica de Gauss
```

```
metodo_formula <- function() {
  n <- 1e10
  suma <- n * (n + 1) / 2
  return(suma)
}
```

```
# Medir tiempo metodo for (advertencia: puede tardar mucho)
```

```
# Para prueba rapida usar 1:1e6 en lugar de 1:1e10
```

```
tiempo_for <- system.time({
  resultado_for <- metodo_for()
})
```

```
# Medir tiempo metodo formula
```

```
tiempo_formula <- system.time({
  resultado_formula <- metodo_formula()
})
```

```
# Mostrar resultados
```

```
print(resultado_for)
```

```
## [1] 5e+19
```

```
print(resultado_formula)
```

```
## [1] 5e+19
```

```
# Mostrar tiempos
```

```
print(tiempo_for)
```

```
##      user  system elapsed
```

```
## 175.193   0.159 176.433
```

```
print(tiempo_formula)
```

```
##      user  system elapsed  
##         0         0         0
```

El algoritmo Bubble Sort (Burbuja) tiene una complejidad computacional de $O(n^2)$. Esto significa que si duplicamos la cantidad de datos, el tiempo de procesamiento se cuadruplica. Es un método “pesado” porque realiza múltiples pasadas e intercambios manuales. Por otro lado, la función `sort()` de R utiliza algoritmos avanzados (como Quicksort o Shellsort) con una complejidad de O elevado a la 2, además de estar implementada en lenguaje C, lo que la hace órdenes de magnitud más eficiente.

```
library(microbenchmark)  
library(ggplot2)
```

```
# Implementación simple de bubble sort
```

```
bubble_sort <- function(x) {  
  n <- length(x)  
  for (i in 1:(n-1)) {  
    for (j in 1:(n-i)) {  
      if (x[j] > x[j+1]) {  
        temp <- x[j]  
        x[j] <- x[j+1]  
        x[j+1] <- temp  
      }  
    }  
  }  
  return(x)  
}
```

```
# Datos de prueba
```

```
set.seed(123)  
vec <- sample(1:1000, 100)
```

```
# Benchmark
```

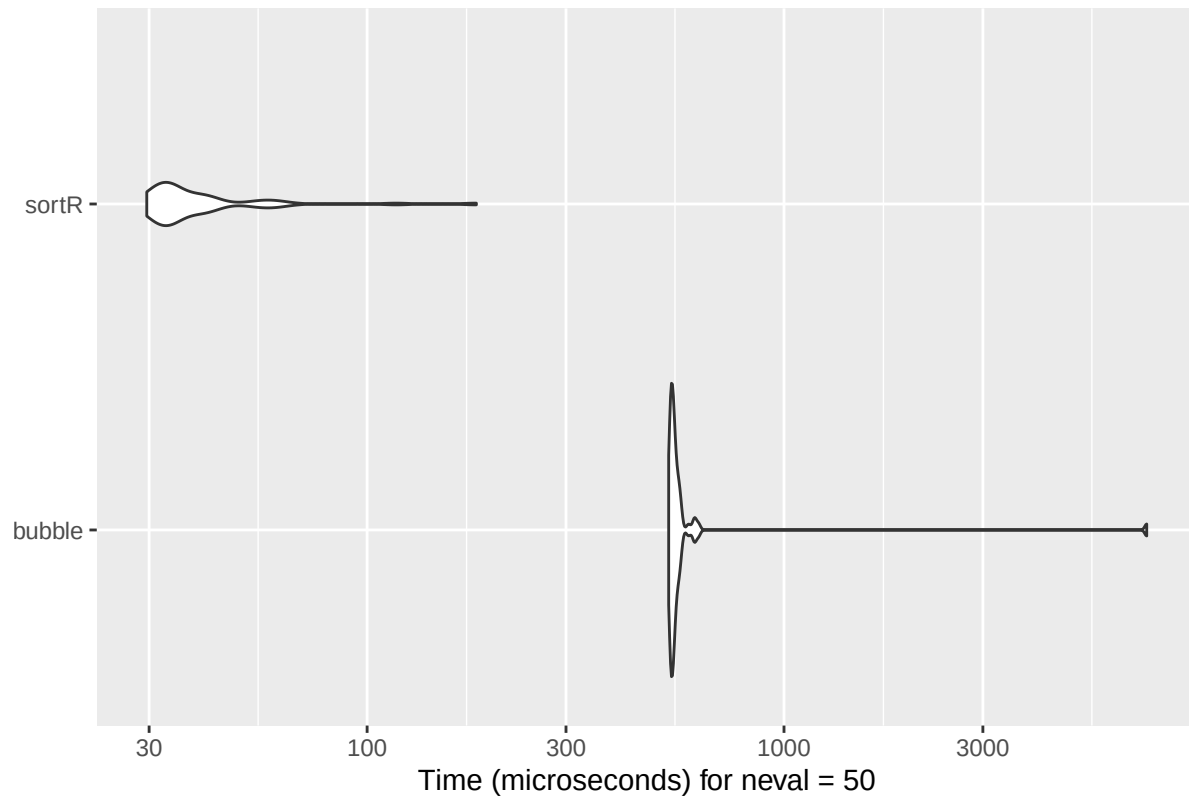
```
bench <- microbenchmark(  
  bubble = bubble_sort(vec),  
  sortR = sort(vec),  
  times = 50  
)
```

```
# Gráfico de violín
```

```
autoplot(bench)
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.  
## i Please use tidy evaluation idioms with `aes()`.  
## i See also `vignette("ggplot2-in-packages")` for more information.  
## i The deprecated feature was likely used in the microbenchmark package.  
## Please report the issue at  
## <https://github.com/joshuaulrich/microbenchmark/issues/>.  
## This warning is displayed once per session.  
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was  
## generated.
```

microbenchmark timings



El K-means es un algoritmo de particionamiento de datos (Clustering) que agrupa observaciones en k grupos basándose en la distancia a un centroide. Es un proceso iterativo que: 1. Define k centros iniciales. 2. Asigna cada punto al centroide más cercano (distancia euclidiana). 3. Recalcula el centroide como el promedio de los puntos asignados. Este ciclo se repite hasta que las asignaciones no cambian o se alcanza un límite de iteraciones. Es ampliamente usado en ingeniería para segmentación de mercado, detección de anomalías y compresión de datos.

```
library(microbenchmark)
library(ggplot2)

set.seed(123)
data <- matrix(rnorm(1000), ncol=2)

bench_kmeans <- microbenchmark(
  kmeans2 = kmeans(data, centers=2),
  kmeans5 = kmeans(data, centers=5),
  times = 50
)

autoplot(bench_kmeans)
```

